Guía de Actividades Práctico-Experimentales Nro. 003

1. Datos Generales

Nombre del estudiante(s)	Israel Rodríguez Douglas Carreño Jossibel Perez Jorge Luzuriaga
Asignatura	Sistemas distribuidos
Ciclo	6to
Unidad	1
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	003
Título de la Práctica	Sincronización de Procesos: Exclusión Mutua, Semáforos y Problemas Clásicos de Concurrencia
Nombre del Docente	José O. Guamán Q.
Fecha	06 de Octubre de 2026 09 de Octubre de 2026
Horario	Martes 10h30 – 11h30 VIernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Implementar una barrera de sincronización con variables de condición que detenga a los hilos hasta que todos alcancen el punto de encuentro.

- Implementar mecanismos de exclusión mutua utilizando mutex y cerrojos (locks) para proteger secciones críticas en programas multihilo, garantizando que no se produzcan condiciones de carrera al acceder a recursos compartidos.

- Aplicar semáforos como herramienta de sincronización para controlar el acceso a recursos limitados y coordinar la ejecución de hilos según restricciones específicas de orden y disponibilidad.
- Resolver el problema clásico del Productor-Consumidor utilizando buffers compartidos y mecanismos de sincronización que aseguren que el productor no escriba en un buffer lleno y el consumidor no lea de un buffer vacío.
- Implementar la solución al problema de Lectores-Escritores, distinguiendo entre las variantes con prioridad para lectores y prioridad para escritores, analizando las implicaciones de cada enfoque en términos de inanición y concurrencia.

3. Materiales, Reactivos:

Guia practica

Lista de ejercicios

- Hardware:
  - Computadora personal con conexión a internet.
- Software:
  - Procesador de texto.

4. Equipos y Herramientas

Hardware
- Computador

Software
- Lenguaje de programación (Python, java sprint boot, node)
- IDE de programación
- Archivo de texto

5. Procedimiento / Metodología

Ejercicio 1. Control de acceso a un estacionamiento — Semáforo contador

Objetivo: Aplicar semáforos contadores para controlar el acceso concurrente a un recurso limitado.

Situación:

Un estacionamiento dispone de 4 espacios. Llegan 10 vehículos de manera concurrente. Cada vehículo debe esperar si no existe un espacio disponible. Cuando un vehículo sale, libera el espacio para otro vehículo.

Procedimiento

1. Identifique el recurso compartido.
2. Determine la cantidad máxima de vehículos que pueden permanecer dentro del estacionamiento.
3. Cree un semáforo contador inicializado en 4.
4. Cree un proceso vehiculo(id).
5. El vehículo deberá solicitar un espacio antes de ingresar.

6. Si no existen espacios disponibles, deberá permanecer bloqueado.
7. Una vez que obtenga un espacio, deberá simular su permanencia.
8. Al salir, deberá liberar el espacio mediante signal().
9. Cree 10 procesos vehículo.
10. Analice qué sucede simultáneamente.

Pseudocódigo

SEMÁFORO espacios = 4

PROCEDIMIENTO vehiculo(id)

escribir("Vehículo ", id, " llega al estacionamiento")

wait(espacios)

escribir("Vehículo ", id, " ingresa al estacionamiento")

estacionar(id)

esperar_tiempo()

escribir("Vehículo ", id, " sale del estacionamiento")

signal(espacios)

FIN PROCEDIMIENTO

INICIO

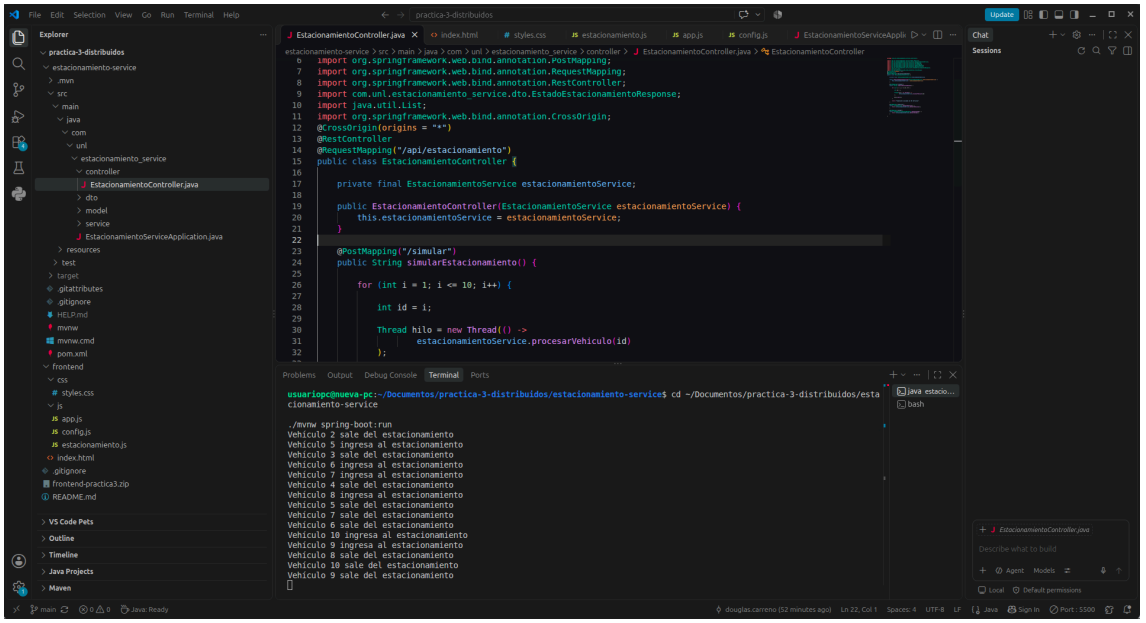
PARA i = 1 HASTA 10 HACER

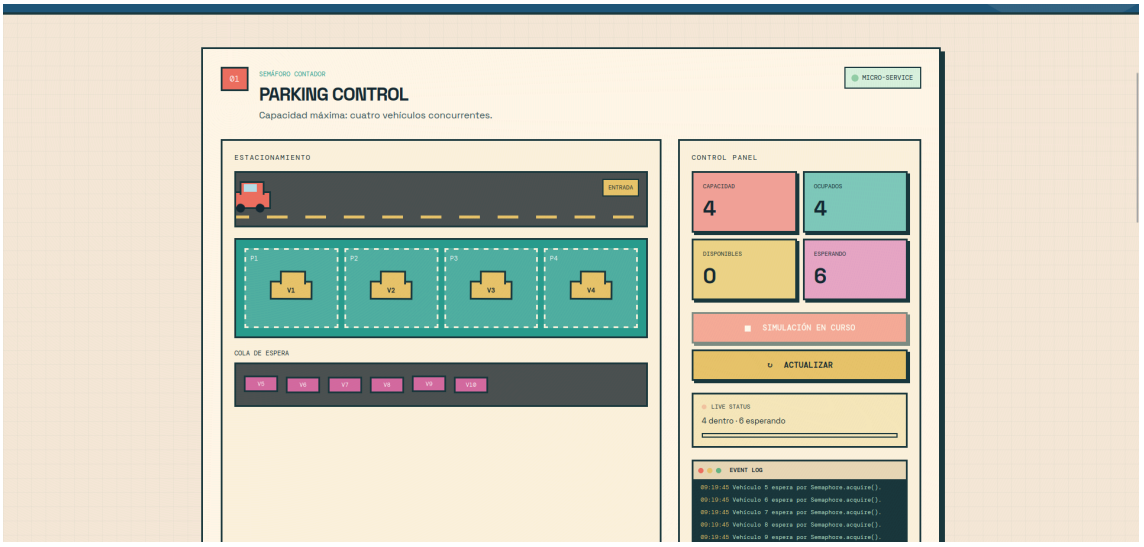
crear_proceso(vehiculo, i)

FIN PARA

FIN

Implementación:





Pregunta de análisis:

¿Qué ocurriría si el semáforo se inicializará en 1? ¿Y si se inicializará en 10?

Si el semáforo se inicializara en 1, el estacionamiento permitiría el ingreso de un solo vehículo a la vez, aunque existan cuatro espacios disponibles. Por lo tanto, los otros nueve vehículos tendrían que esperar hasta que el vehículo que está dentro salga y libere el permiso. Esto garantizaría el control de acceso, pero reduciría innecesariamente la concurrencia y el aprovechamiento de los espacios.

En cambio, si se inicializara en 10, el semáforo permitiría el ingreso simultáneo de los diez vehículos, a pesar de que el estacionamiento únicamente dispone de cuatro espacios. Esto provocaría que el control de acceso no respete la capacidad real del recurso, permitiendo una sobreocupación en la simulación.

Por esta razón, el valor correcto es 4, ya que permite aprovechar todos los espacios disponibles sin superar la capacidad del estacionamiento.

Ejercicio 2. Cajero automático compartido — Exclusión mutua

Objetivo: Implementar exclusión mutua para evitar condiciones de carrera sobre un recurso compartido.

Situación:

Una cuenta bancaria tiene un saldo inicial de $1000. Tres procesos realizan operaciones simultáneamente sobre la misma cuenta:

- Proceso 1: retiro de $300.
- Proceso 2: retiro de $250.
- Proceso 3: depósito de $200.

El saldo debe modificarse de forma segura.

Procedimiento

1. Identifique la variable compartida.
2. Determine qué operaciones pueden ejecutarse simultáneamente.
3. Cree un mutex para proteger el saldo.
4. Antes de consultar o modificar el saldo, el proceso debe adquirir el mutex.
5. Realice la operación correspondiente.
6. Libere el mutex después de actualizar el saldo.
7. Cree los tres procesos.
8. Ejecute el programa varias veces.
9. Analice qué podría ocurrir si se elimina el mutex.
10. Compare el comportamiento con y sin exclusión mutua.

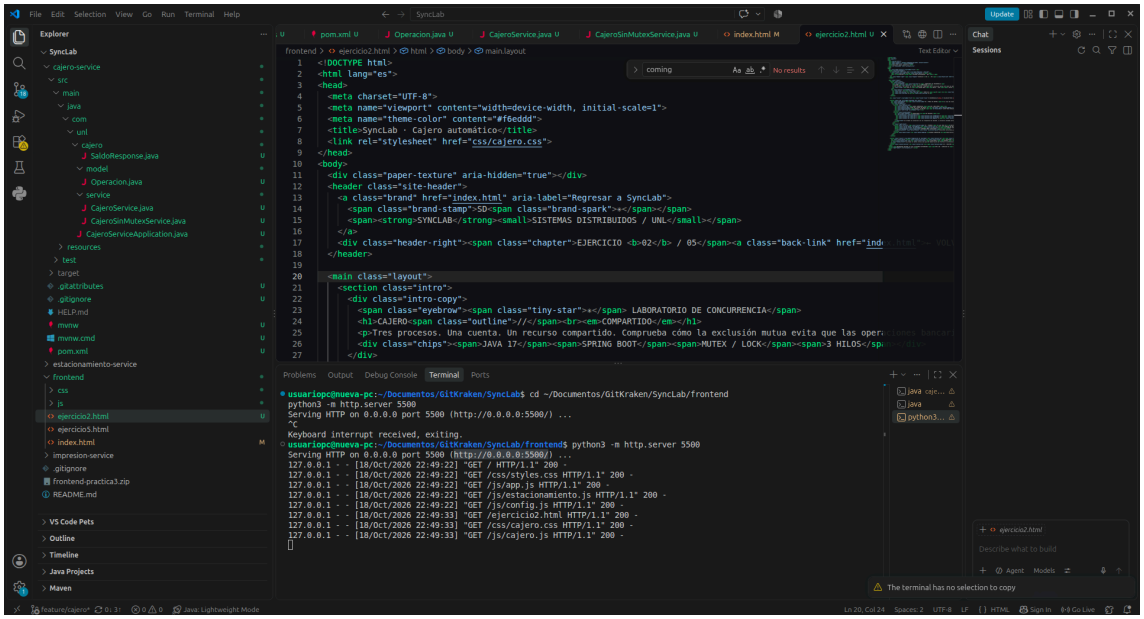
Pseudocódigo

```
saldo = 1000
MUTEX mutex_saldo = 1
PROCEDIMIENTO retirar(id, cantidad)
   lock(mutex_saldo)
   escribir("Proceso ", id, " intenta retirar $", cantidad)
   SI saldo >= cantidad ENTONCES
      saldo = saldo - cantidad
      escribir("Retiro realizado")
      escribir("Saldo actual: $", saldo)
   SINO
      escribir("Saldo insuficiente")
   FIN SI
   unlock(mutex_saldo)
FIN PROCEDIMIENTO
```

```
PROCEDIMIENTO depositar(id, cantidad)
    lock(mutex_saldo)
    escribir("Proceso ", id, " deposita $", cantidad)
    saldo = saldo + cantidad
    escribir("Saldo actual: $", saldo)
    unlock(mutex_saldo)
FIN PROCEDIMIENTO

INICIO
    crear_proceso(retirar, 1, 300)
    crear_proceso(retirar, 2, 250)
    crear_proceso(depositar, 3, 200)
FIN
```

Implementación:





Pregunta de análisis:

¿Por qué el acceso a saldo debe protegerse con un mutex?

El acceso al saldo debe protegerse mediante un mutex porque es una variable compartida que puede ser consultada y modificada por varios hilos concurrentemente. Sin exclusión mutua, dos o más procesos podrían leer el mismo saldo antes de que otro termine su operación, provocando condiciones de carrera y pérdida de actualizaciones.

Esto se comprobó en nuestra implementación del cajero automático, donde tres procesos realizaron dos retiros de $300 y $250, y un depósito de $200 sobre un saldo inicial de $1000. Al utilizar ReentrantLock, el resultado final fue $650, mientras que en una ejecución sin mutex se obtuvo $1200, debido a que las operaciones se sobrescribieron.

Por lo tanto, el mutex permite que únicamente un hilo acceda a la sección crítica a la vez, garantizando que las modificaciones del saldo se realicen de manera segura y consistente.

Ejercicio 3. Fábrica de ensamblaje — Productor-Consumidor

Objetivo: Implementar el patrón productor-consumidor utilizando una cola limitada y semáforos.

Situación:

Una fábrica tiene una línea de producción con capacidad para almacenar 5 piezas temporalmente.

- Los productores fabrican piezas.
- Los consumidores toman piezas para ensamblarlas.
- Un productor no puede colocar una pieza si el almacén está lleno.
- Un consumidor no puede retirar una pieza si el almacén está vacío.

Procedimiento:

1. Cree una cola con capacidad máxima de 5 elementos.
2. Cree un semáforo espacios inicializado en 5.
3. Cree un semáforo elementos inicializado en 0.
4. Cree un mutex para proteger la cola.

5. Implemente el proceso productor.
6. El productor deberá esperar un espacio disponible.
7. Debe adquirir el mutex antes de modificar la cola.
8. Debe insertar una pieza.
9. Debe liberar el mutex.
10. Debe indicar que existe una nueva pieza.
11. Implemente el proceso consumidor.
12. El consumidor deberá esperar hasta que exista una pieza.
13. Debe adquirir el mutex antes de retirar una pieza.
14. Debe retirar la pieza.
15. Debe liberar el mutex.
16. Debe indicar que existe un espacio disponible.

Pseudocódigo

```
COLA almacen
CAPACIDAD = 5

SEMÁFORO espacios = 5
SEMÁFORO elementos = 0

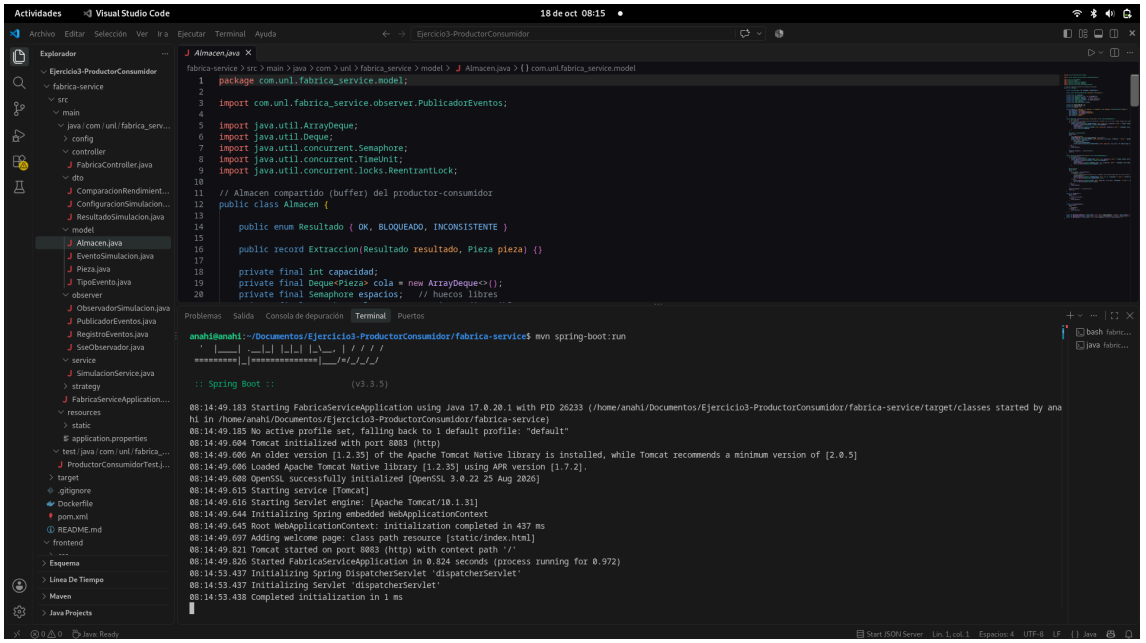
MUTEX mutex_almacen = 1
PROCEDIMIENTO productor(id)
   REPETIR 5 VECES
      pieza = fabricar_pieza()
      wait(espacios)
      lock(mutex_almacen)
      insertar(almacen, pieza)
      escribir("Productor ", id,
            " agregó pieza ", pieza)
      unlock(mutex_almacen)
      signal(elementos)
   FIN REPETIR
FIN PROCEDIMIENTO

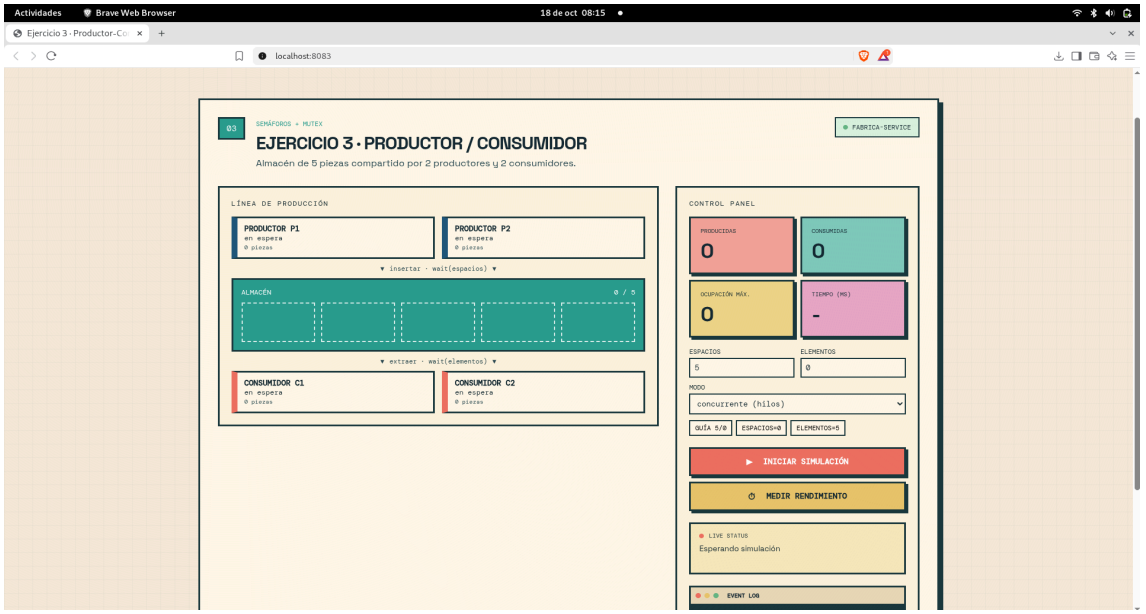
PROCEDIMIENTO consumidor(id)
   REPETIR 5 VECES
      wait(elementos)
      lock(mutex_almacen)
      pieza = extraer(almacen)
      escribir("Consumidor ", id,
```

```
            " retiró pieza ", pieza)
        unlock(mutex_almacen)
        signal(espacios)
        ensamblar(pieza)
    FIN REPETIR
FIN PROCEDIMIENTO

INICIO
    PARA i = 1 HASTA 2 HACER
        crear_proceso(productor, i)
    FIN PARA
    PARA i = 1 HASTA 2 HACER
        crear_proceso(consumidor, i)
    FIN PARA
FIN
```

Implementación:





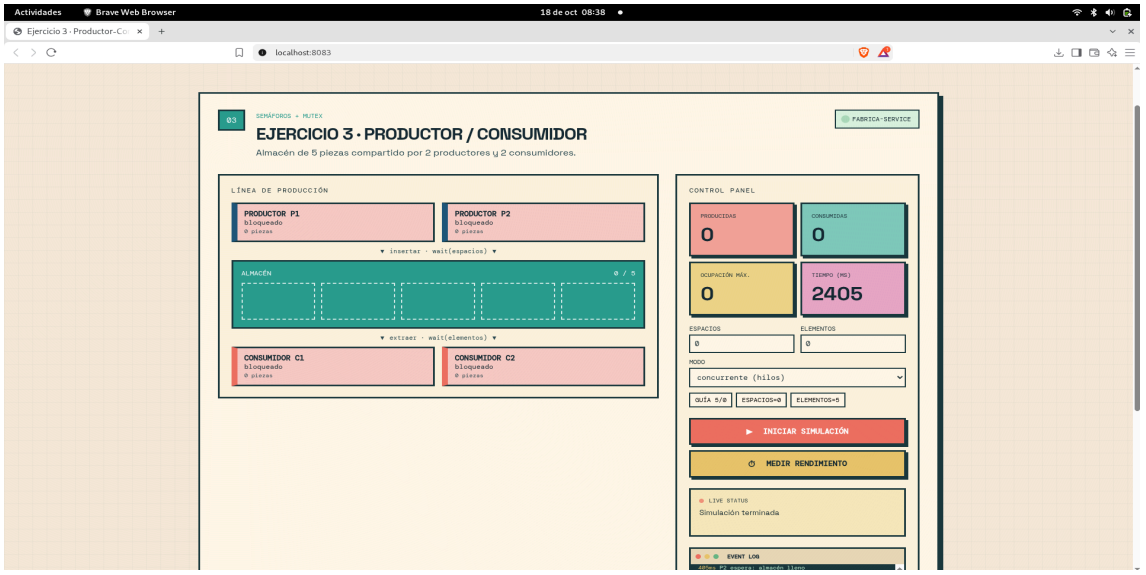
Pregunta de análisis:

¿Qué sucede si espacios se inicializa en 0?

Si se inicia desde 0 se produce un interbloqueo en el que se detiene nadie puede avanzar, el semáforo espacio nos dice cuántos espacios libre hay en el almacenamiento, por eso si se inicia desde 0 el sistema asume que el almacenamiento está lleno, aunque esté vacío.

- Los productores ejecutan wait(espacios) y se bloquean, esto porque no hay lugar libre. Así que nunca insertan una pieza.
- Los consumidores ejecutan wait(elementos) y estos también se bloquean, debido a que los elementos siguen en 0 y no hay llegada de ninguna pieza.
- Cada uno de los grupos espera algo que solo el otro grupo puede darles, así todos quedan totalmente separados.

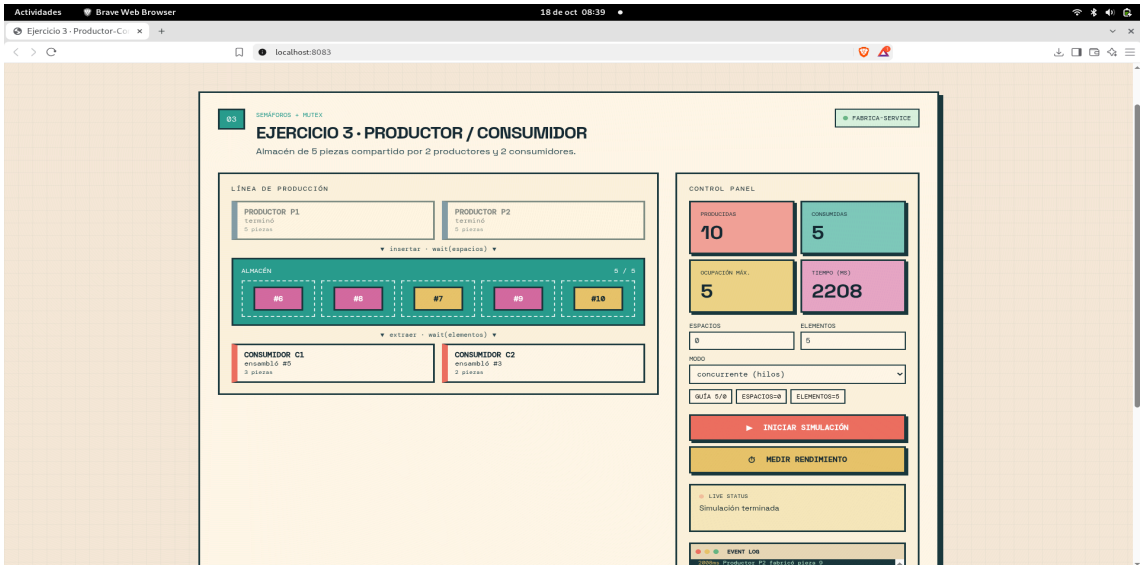
Resultado: 0 piezas producidas, 0 consumidas y los 4 hilos bloqueados.
En la página, los cuatro ejecutantes se ponen en rojo con el mensaje “bloqueado”.



¿Qué sucede si elementos se inicializa en 5?

Se produce una inconsistencias, ya que el semáforo elementos dice cuántas piezas hay para consumirse y si se empieza con 5 el sistema cree automáticamente que hay 5 piezas pero no es verdad ya que el almacenamiento está vacío

Los consumidores pasan wait(elementos) sin esperar y van a retirar piezas que no existen , es decir leen un almacén vacío y cada retiro falso ejecuta signal (espacios). Eso hace que se sumen espacios de más, y los productores llegan a aumentar más de 5 piezas, pasándose la capacidad de almacenamiento.



Al final no se consumen todas las piezas producidas.

Resultado: 10 producidas, solo 5 consumidas, entre 5 y 7 inconsistencias, y con una ocupación máxima que a veces llega a 6 o 7 piezas con una capacidad de 5. Los números cambian en cada una de las ejecuciones, porque dependen mucho del orden en que el sistema operativo ejecuta los hilos.

Ejercicio 4. Cruce de vehículos — Control de acceso en una dirección

Objetivo: Coordinar procesos que comparten un recurso y evitar que dos grupos incompatibles accedan simultáneamente.

Situación:

Un puente permite circular en ambos sentidos, pero solamente puede existir **una dirección de circulación activa a la vez.**

Pueden existir varios vehículos cruzando hacia el norte, pero no puede haber vehículos hacia el norte y hacia el sur simultáneamente.

Procedimiento:

1. Identifique el recurso compartido.
2. Defina una variable que indique la dirección actual.
3. Mantenga un contador de vehículos que están cruzando.
4. Proteja las variables mediante un mutex.
5. Un vehículo deberá verificar la dirección antes de entrar.
6. Si existen vehículos en dirección contraria, deberá esperar.
7. Si puede entrar, incrementará el contador correspondiente.
8. Después de cruzar, disminuirá el contador.
9. Cuando no queden vehículos, el puente podrá cambiar de dirección.
10. Analice el problema de inanición.

Pseudocódigo

```
MUTEX mutex = 1
direccion = NINGUNA
vehiculos_norte = 0
vehiculos_sur = 0
PROCEDIMIENTO vehiculo_norte(id)
 lock(mutex)
 MIENTRAS direccion == SUR HACER
  esperar()
 FIN MIENTRAS
 direccion = NORTE
 vehiculos_norte = vehiculos_norte + 1
 unlock(mutex)
 escribir("Vehículo ", id,
 " cruza hacia el NORTE")
 cruzar_puente(id)
```

```
  lock(mutex)
  vehiculos_norte = vehiculos_norte - 1
  SI vehiculos_norte == 0 ENTONCES
    direccion = NINGUNA
  FIN SI
  unlock(mutex)
FIN PROCEDIMIENTO
```

El estudiante deberá completar el procedimiento:
PROCEDIMIENTO vehiculo_sur(id)
siguiendo la misma lógica.
Reto: modificar el algoritmo para evitar que una dirección monopolice el puente.

```
MUTEX mutex = 1
CONDICION condicion
direccion = NINGUNA
vehiculos_norte = 0
esperando_norte = 0
esperando_sur = 0
consecutivos = 0
turno = NINGUNA
LIMITE = 3

PROCEDIMIENTO vehiculo_norte(id)
    lock(mutex)
    esperando_norte = esperando_norte + 1
    MIENTRAS NO puede_entrar(NORTE) HACER
        esperar(condicion)            // suelta el mutex mientras espera
    FIN MIENTRAS
    esperando_norte = esperando_norte - 1
    SI direccion != NORTE ENTONCES
        direccion = NORTE
        consecutivos = 0
    FIN SI
    consecutivos = consecutivos + 1
```

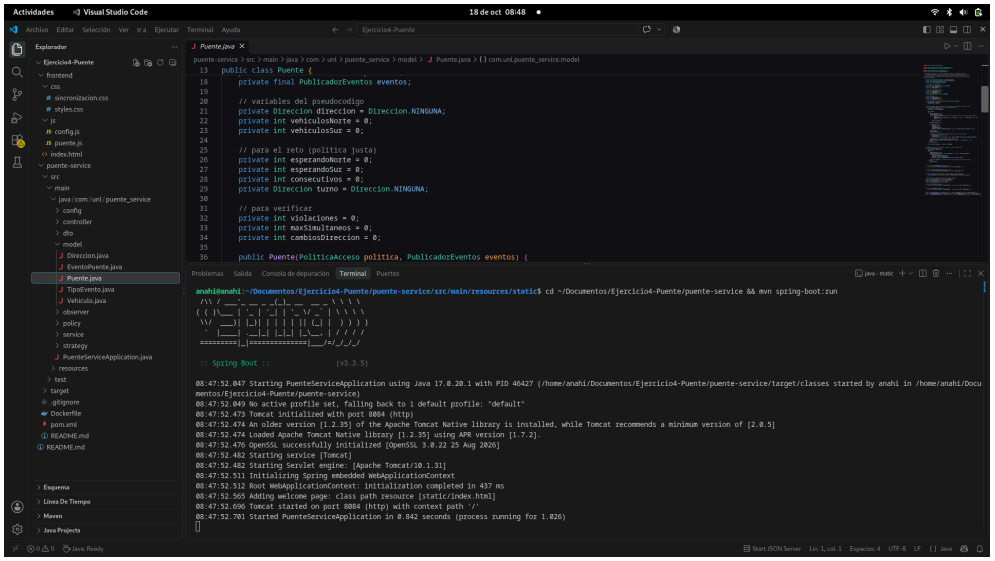
```
    vehiculos_norte = vehiculos_norte + 1
    unlock(mutex)

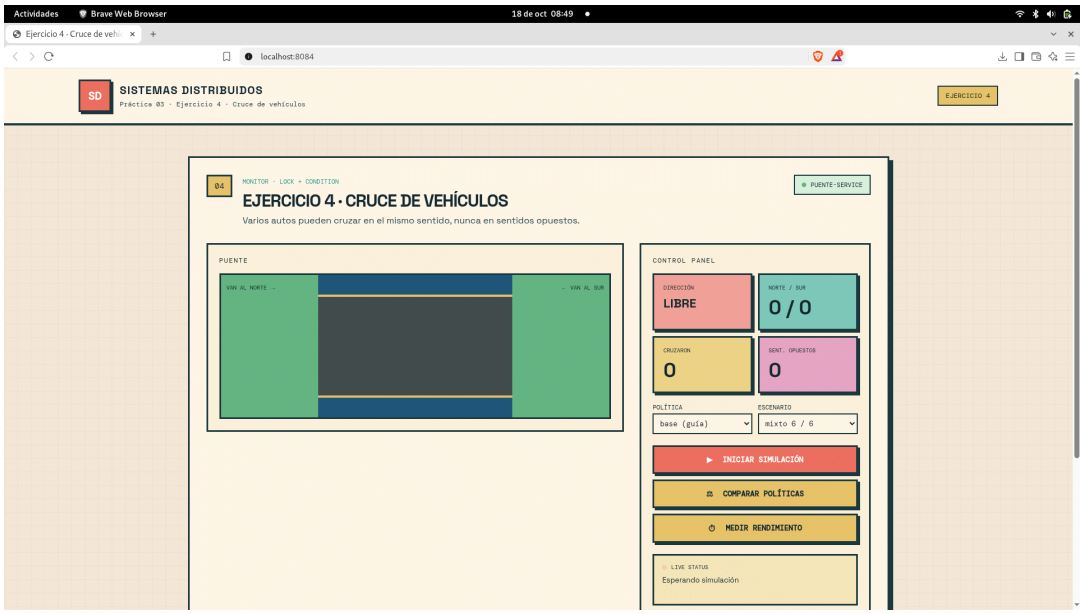
    cruzar_puente(id)

    lock(mutex)
    vehiculos_norte = vehiculos_norte - 1
    SI vehiculos_norte == 0 ENTONCES
        SI esperando_sur > 0 ENTONCES turno = SUR SINO turno = NINGUNA
        direccion = NINGUNA
    FIN SI
    despertar_a_todos(condicion)
    unlock(mutex)
FIN PROCEDIMIENTO

FUNCION puede_entrar(d)
    SI direccion == NINGUNA ENTONCES
                    RETORNAR turno == NINGUNA O turno == d O
nadie_espera_enfrente(d)
    SI direccion == d ENTONCES
        RETORNAR nadie_espera_enfrente(d) O consecutivos < LIMITE
    RETORNAR FALSO                  // va en sentido contrario
FIN FUNCION
```

Implementación:





Ejercicio 5. Centro de impresión — Sincronización de procesos

Objetivo: Utilizar semáforos para establecer un orden determinado de ejecución entre procesos.

Situación:

Un centro de impresión tiene tres procesos:

- Diseño
- Revisión
- Impresión

Para cada documento debe cumplirse obligatoriamente:

DISEÑO → REVISIÓN → IMPRESIÓN

No se puede revisar un documento que todavía no ha sido diseñado y tampoco se puede imprimir antes de que haya sido revisado.

Procedimiento:

1. Identifique los procesos participantes.
2. Cree un semáforo para cada etapa.
3. Permita inicialmente que el proceso de diseño pueda ejecutarse.
4. Diseño deberá habilitar a revisión.
5. Revisión deberá habilitar a impresión.
6. Impresión deberá habilitar nuevamente a diseño.
7. Repita el ciclo para 3 documentos.
8. Verifique que el orden siempre sea correcto.
9. Analice qué ocurre si se elimina uno de los signal().

Pseudocódigo

SEMÁFORO turno_diseno = 1

SEMÁFORO turno_revision = 0

SEMÁFORO turno_impresion = 0

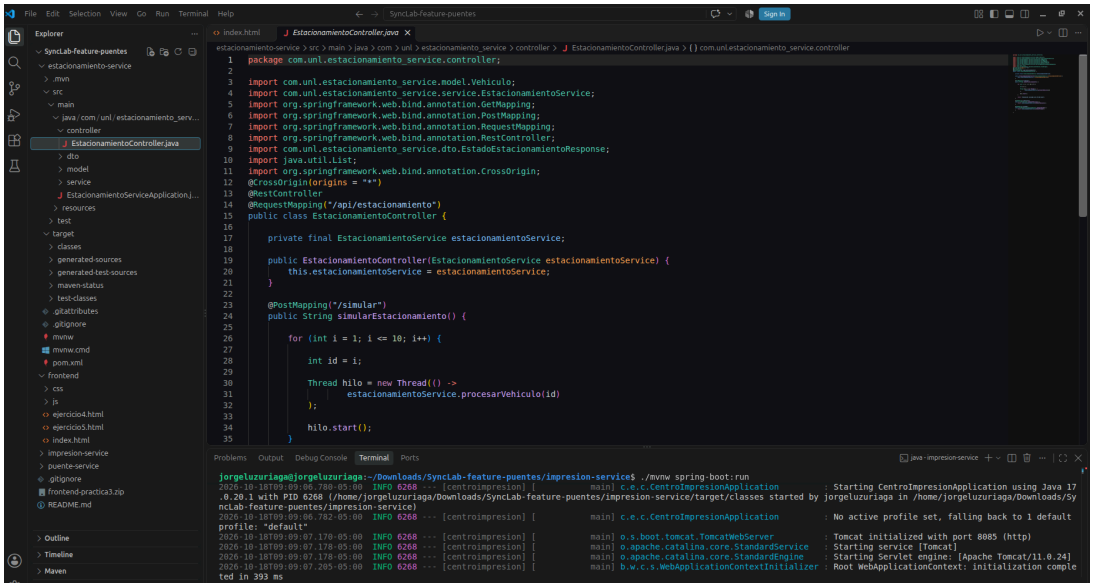
```
PROCEDIMIENTO diseno()
  REPETIR 3 VECES
     wait(turno_diseno)
     escribir("Diseño prepara documento")
     crear_documento()
     signal(turno_revision)
  FIN REPETIR
FIN PROCEDIMIENTO

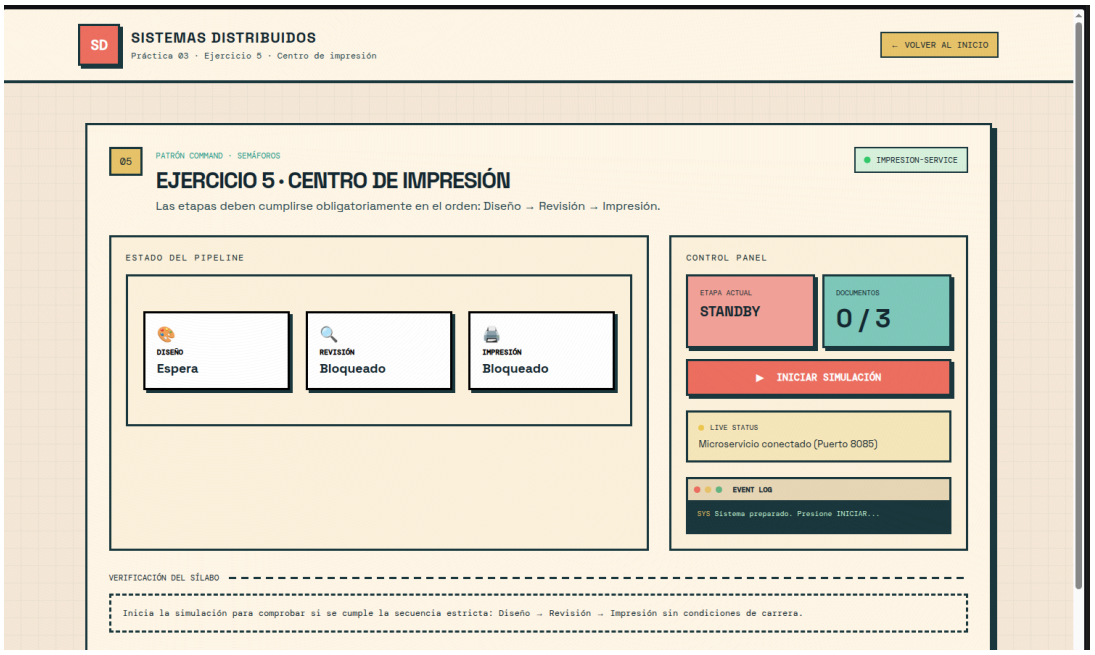
PROCEDIMIENTO revision()
  REPETIR 3 VECES
     wait(turno_revision)
     escribir("Revisión verifica documento")
     revisar_documento()
     signal(turno_impresion)
  FIN REPETIR
FIN PROCEDIMIENTO

PROCEDIMIENTO impresion()
  REPETIR 3 VECES
     wait(turno_impresion)
     escribir("Impresión imprime documento")
     imprimir_documento()
     signal(turno_diseno)
  FIN REPETIR
FIN PROCEDIMIENTO

INICIO
  crear_proceso(diseno)
  crear_proceso(revision)
  crear_proceso(impresion)
FIN
```

Implementación:





6. Resultados esperados:

Al finalizar la práctica, el estudiante deberá haber obtenido y documentado lo siguiente:

- Comprobación de las distintas formas de cómo los hilos se comunican.
- Se espera que el estudiante mida y compare el rendimiento entre las soluciones secuenciales y concurrentes.
- El estudiante debe demostrar comprensión de los conceptos vistos en cada ejercicio de la práctica.

Enlace al repositorio en Github.

Criterios de verificación:

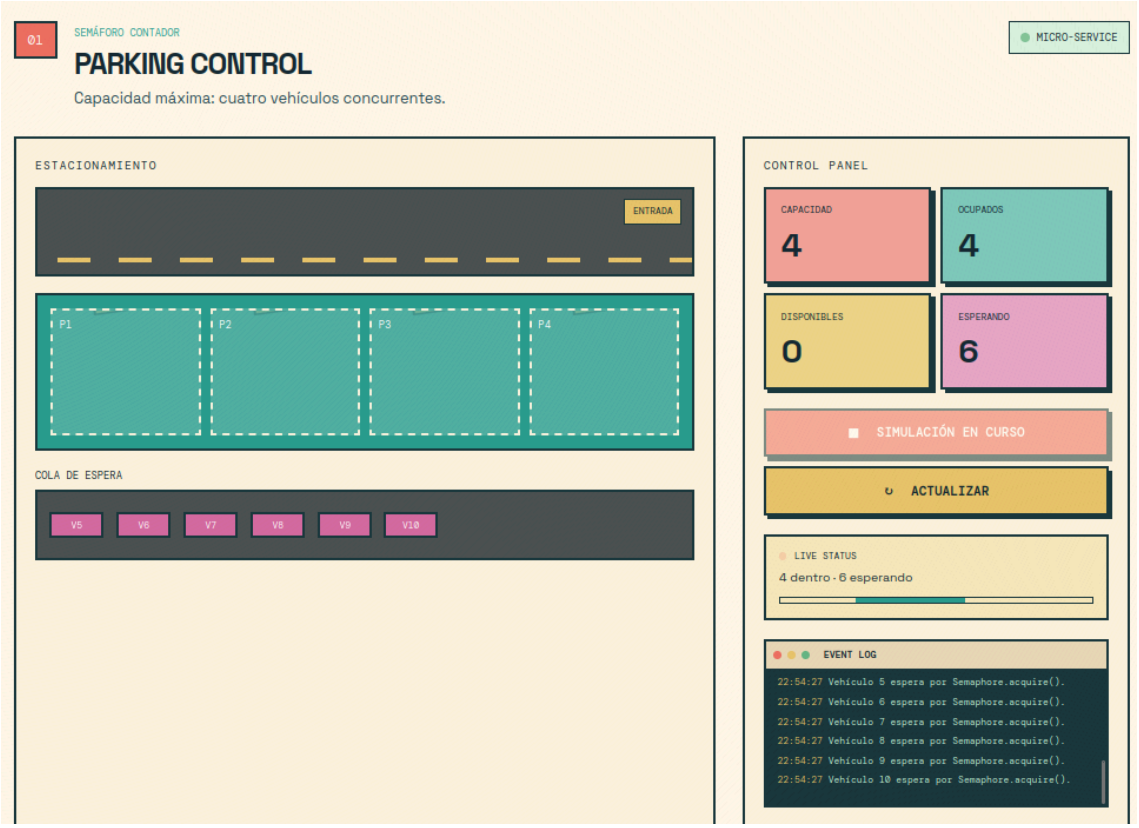


Figura 1: Verificación de que nunca ingresen más de 4 vehículos simultáneamente.



Figura 2: Verificación de que el saldo no varíe en cada ejecución.

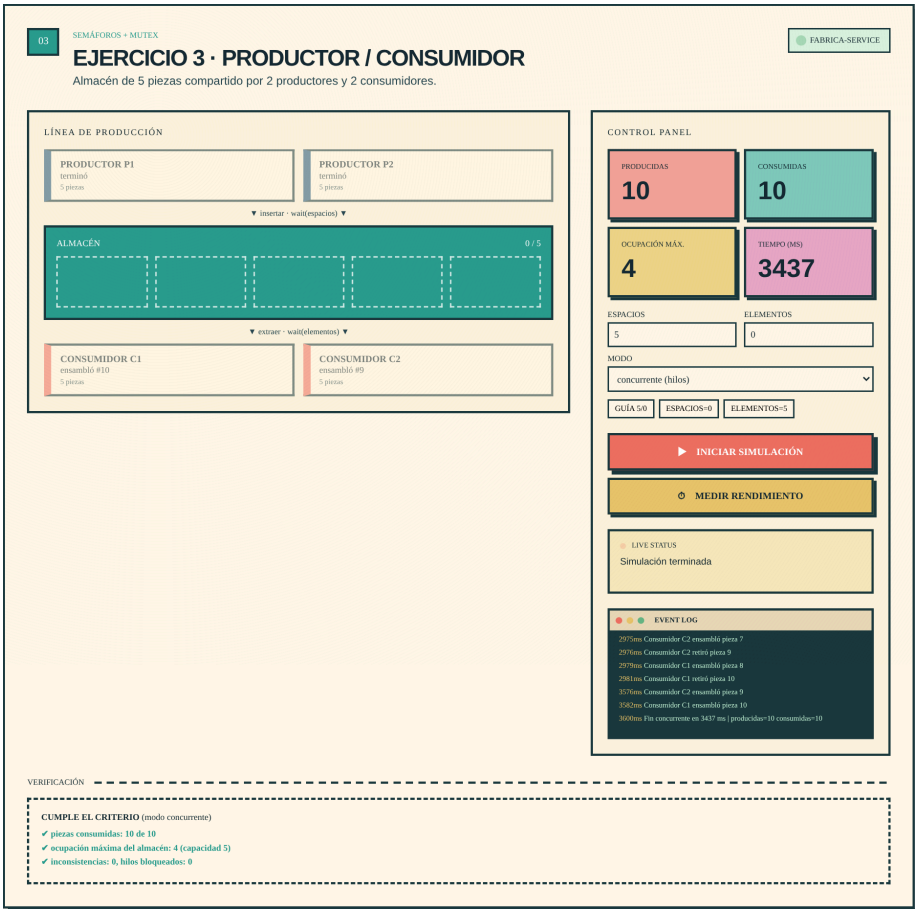


Figura 3: Verificación de que el almacén nunca supere 5 productos, ni quede con inconsistencias.

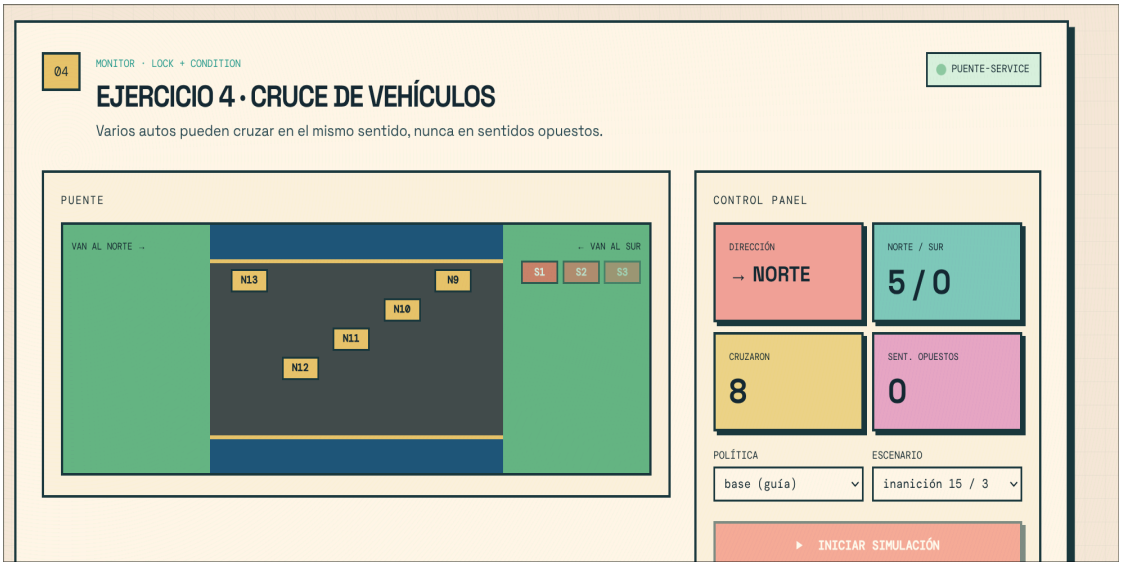


Figura 4: Verificación de que nunca crucen vehículos en sentidos opuestos simultáneamente.

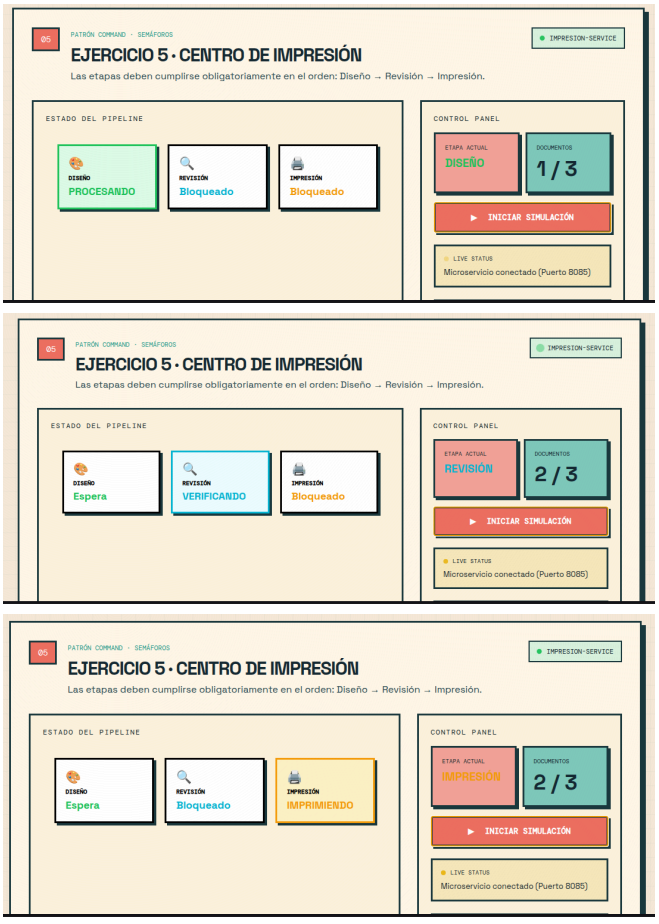


Figura 5: Verificación de que ningún proceso avance a la siguiente etapa antes de tiempo

7. Conclusiones:

- Se logró implementar la barrera de sincronización mediante variables de condición para detener los hilos hasta que todos alcancen el punto de encuentro.
- Se aplicaron con éxito mecanismos de exclusión mutua con mutex y cerrojos para proteger las secciones críticas y así evitar las condiciones de carrera.
- Se logró aplicar el uso de semáforos para regular el acceso a recursos limitados y coordinar la ejecución según las restricciones de orden y disponibilidad.
- Se pudo resolver el problema del Productor-Consumidor garantizando la sincronización adecuada sobre el buffer compartido sin producir sobreescrituras ni lecturas en vacío.
- Se logró implementar la solución al problema de Lectores-Escritores, identificando las diferencias entre priorizar lectores o escritores en términos de inanición y concurrencia.

8. Preguntas de Control

- **¿Qué es una condición de carrera?**

  Una condición de carrera es cuando dos o más hilos buscan acceder y modificar un recurso compartido de forma simultánea, lo cual es crítico en un sistema ya que si un hilo es interrumpido antes de completar por ejemplo una escritura, otro hilo puede sobreescribir y distorsionar la información que existía en un principio [1].

- **Cuál es la diferencia fundamental entre un semáforo binario y un mutex? ¿Pueden usarse indistintamente? Justifique su respuesta con al menos dos diferencias prácticas.**

  La diferencia fundamental reside en el propósito, ya que si hablamos del Mutex este está diseñado exclusivamente para la protección de recursos compartidos en donde solo el hilo que tomó el mutex está autorizado a liberarlo. Por su parte, el semáforo binario está diseñado para la sincronización de eventos entre hilos [2].

  En cuanto a si se los puede usar indistintamente, la respuesta es no, ya que ambos internamente funcionan de manera distinta y al emplearlos de manera cruzada se puede vulnerar la seguridad del sistema y generar fallos graves e impredecibles en la ejecución de los hilos, como el liberar un recurso por error, o que exista un deadlock [2].

  Como diferencias prácticas se puede resaltar el apartado de propiedad y liberación, ya que en un mutex el hilo que usa un recurso debe obligatoriamente liberarlo cuando termina de usarlo, cosa que en los semáforos no pasa ya que no existe el concepto de propiedad. Asimismo, un diferencia práctica es el mecanismo de Herencia de prioridad que usan los mutexes, donde si una tarea de alta prioridad se bloquea esperando un mutex retenido por una tarea de baja prioridad, el sistema eleva temporalmente la prioridad de esta última para que termine rápido y libere el recurso, cosa que los semáforos no lo contemplan [2].

- **Explique qué es un interbloqueo (deadlock) y enuncie las cuatro condiciones necesarias de Coffman. ¿Cuál de estas condiciones se elimina al usar la estrategia de ordenamiento de recursos?**

  Un interbloqueo es una situación en la que dos o más tareas quedan bloqueadas de manera indefinida sin poder continuar, debido a que cada una está esperando un recurso que está retenido por la otra [2].

  Y para que ocurra ello, deben cumplirse las cuatro condiciones de Coffman [3], las cuales son:

  1. Exclusión Mutua: Al menos un recurso debe estar asignado a una sola tarea a la vez, de forma que el recurso no sea compartido mientras se usa.

  2. Retención y espera: Una tarea posee o retiene al menos un recurso y, al mismo tiempo, está solicitando recursos adicionales que actualmente están retenidos por otras tareas..

  3. No apropiación: Los recursos no pueden ser quitados por la fuerza a las tareas que lo usan, de modo que solo la tarea que lo está usando puede liberarlo voluntariamente tras finalizar su trabajo

  4. Espera Circular: Se refiere a que existe una cadena cerrada de tareas en la que cada una sostiene un recurso y espera otro recurso retenido por la siguiente tarea de la cadena, estando la última tarea a la espera del recurso retenido por la primera.

  En cuanto al aplicar la estrategia de ordenamiento de recursos, se elimina la condición de espera circular, ya que al no permitir que una tarea solicite un recurso de orden inferior si ya posee uno de orden superior, resulta imposible que se forme un ciclo de dependencias circulares entre las tareas [3].

9. Declaración de uso de IA

Para el desarrollo de la presente práctica se emplearon herramientas de Inteligencia Artificial con fines de apoyo académico y técnico. En particular, se utilizó NotebookLM como recurso para facilitar el entendimiento de la temática y la estructura conceptual de la práctica.

Además se utilizó DeepSeek para la facilitación en la creación de código, pero fue revisado y editado por el grupo en base a los requerimientos de la práctica.

Prompt:

“Actúa como un Ingeniero Frontend Senior y Diseñador de UI/UX experto en desarrollo web nativo (Vanilla HTML, CSS moderno con Flexbox/Grid y JavaScript asíncrono).

En base a los requisitos del documento APE3_Guia, necesito crear el frontend para un "Simulador de Concurrencia" de la materia de Sistemas Distribuidos (Práctica 03: Sincronización de procesos). La aplicación debe conectar y coordinar visualmente 5 microservicios independientes.

La interfaz debe ser llamativa y los 5 microservicios/ejercicios que contendrá este frontend unificado son:

01. PARKING CONTROL (Semáforo Contador): Capacidad máxima de 4 vehículos concurrentes. Control de estacionamiento, cola de espera y panel con contadores (Capacidad, Ocupados, Disponibles, Esperando).

02. CAJERO AUTOMÁTICO: Exclusión mutua sobre saldo compartido.

03. PRODUCTOR / CONSUMIDOR: Buffer limitado y sincronización.

04. CRUCE DE VEHÍCULOS: Circulación concurrente por dirección.

05. CENTRO DE IMPRESIÓN: Flujo Distribución -> Revisión -> Impresión.

Requerimientos Técnicos y Estructura:

1. Arquitectura "Vanilla" Limpia: El proyecto debe estructurarse estrictamente en archivos nativos (HTML para las vistas, una carpeta /style o /css para las hojas de estilo unificadas, y una carpeta /js para la lógica de conexión e interactividad).

2. Layout y Navegación: Una página principal (Dashboard / Home) con un Hero Section que explique el simulador ("Procesos que compiten. Recursos que deben coordinarse") y un menú interactivo tipo rejilla (Grid) con tarjetas independientes para acceder o simular cada uno de los 5 microservicios.

3. Consistencia Visual y Estado de Red: La interfaz debe mantener una paleta de colores corporativa (fondos crema/beige suave, acentos en bloques de color verde menta, coral/rojo, azul y amarillo pastel con textos oscuros). Además, cada componente de microservicio debe incluir un indicador visual de estado en tiempo real (ej. "LIVE STATUS: Microservicio no disponible" o "SISTEMA ONLINE").

10. Bibliografía

[1] J. Pagán Ortiz, P. J. Malagón Marzo, R. Cárdenas Rodríguez y J. J. Gómez Valverde, Fundamentos teóricos de sistemas basados en microcontrolador STM32, versión 1.2.2. Madrid, España: Universidad Politécnica de Madrid, Escuela Técnica Superior de Ingenieros de Telecomunicación, Departamento de Ingeniería Electrónica, 2025.

[2] R. Barry y FreeRTOS Team, Mastering the FreeRTOS™ Real Time Kernel: A Hands-On Tutorial Guide, versión 1.1.0. Amazon Web Services, 2023.

[3] M. H. Hassan, S. M. Darwish, and S. M. Elkaffas, “An Efficient Deadlock Handling Model Based on Neutrosophic Logic: Case Study on Real Time Healthcare Database Systems,” IEEE Access, vol. 10, pp. 76607–76621, 2022, doi: 10.1109/ACCESS.2022.3192414.